



Protocol Audit Report

Version 1.0

Cyfrin.io

July 21, 2026

Protocol Audit Report

Cyfrin.io

July 21, 2026

Prepared by: Cyfrin Lead Auditors: - ALtera21

Table of Contents

- Table of Contents
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
- Protocol Summary
 - Roles
 - Issues Found
- High
 - [H-1] `VaultGuardianToken` received by guardian can be minted everytime someone becomes a guardian with no cost
 - [H-2] Miscalculation in `UniswapAdapter::_uniswapInvest` causes allocation data to be wrong
 - [H-3] Ignoring slippage protection `UniswapAdapter::_uniswapInvest` function
- Medium
 - [M-1] `VaultShares::updateHoldingAllocation` did not update the holding allocations automatically
- Questions

Disclaimer

The ALtera21 team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
1 daa885d5cb05b4fe1c48f9708ac1c096b372f3c0
```

Scope

```
1 ./src/  
2 #-- abstract  
3 |   #-- AStaticTokenData.sol  
4 |   #-- AStaticUSDCData.sol  
5 |   #-- AStaticWethData.sol  
6 #-- dao  
7 |   #-- VaultGuardianGovernor.sol  
8 |   #-- VaultGuardianToken.sol  
9 #-- interfaces
```

```
10 |    |-- IVaultData.sol
11 |    |-- IVaultGuardians.sol
12 |    |-- IVaultShares.sol
13 |    |-- InvestableUniverseAdapter.sol
14 |-- protocol
15 |    |-- VaultGuardians.sol
16 |    |-- VaultGuardiansBase.sol
17 |    |-- VaultShares.sol
18 |    |-- investableUniverseAdapters
19 |        |-- AaveAdapter.sol
20 |        |-- UniswapAdapter.sol
21 |-- vendor
22 |    |-- DataTypes.sol
23 |    |-- IPool.sol
24 |    |-- IUniswapV2Factory.sol
25 |    |-- IUniswapV2Router01.sol
```

Protocol Summary

This protocol allows users to deposit certain ERC20s into an ERC4626 vault managed by a human being, or a `vaultGuardian`. The goal of a `vaultGuardian` is to manage the vault in a way that maximizes the value of the vault for the users who have deposited money into the vault.

Roles

There are 4 main roles associated with the system.

- *Vault Guardian DAO*: The org that takes a cut of all profits, controlled by the `VaultGuardianToken`. The DAO that controls a few variables of the protocol, including:
 - `s_guardianStakePrice`
 - `s_guardianAndDaoCut`
 - And takes a cut of the ERC20s made from the protocol
- *DAO Participants*: Holders of the `VaultGuardianToken` who vote and take profits on the protocol
- *Vault Guardians*: Strategists/hedge fund managers who have the ability to move assets in and out of the investable universe. They take a cut of revenue from the protocol.
- *Investors*: The users of the protocol. They deposit assets to gain yield from the investments of the Vault Guardians.

Issues Found

Severity	Number of issues found
High	3
Medium	1
Low	0
Info	0
Gas	0
Total	6

High

[H-1] VaultGuardianToken received by guardian can be minted everytime someone becomes a guardian with no cost

Description: The `VaultGuardianToken` was meant to be minted by the guardian through `VaultGuardiansBase::becomeGuardian` function. This function then calls a private `VaultGuardiansBase::_becomeTokenGuardian` function which mints `VaultGuardianToken` to the guardian

```
1     function _becomeTokenGuardian(IERC20 token, VaultShares tokenVault)
2         private returns (address) {
3         s_guardians[msg.sender][token] = IVaultShares(address(
4             tokenVault));
5         emit GuardianAdded(msg.sender, token);
6         i_vgToken.mint(msg.sender, s_guardianStakePrice);
7         token.safeTransferFrom(msg.sender, address(this),
8             s_guardianStakePrice);
9         bool succ = token.approve(address(tokenVault),
10             s_guardianStakePrice);
11         if (!succ) {
12             revert VaultGuardiansBase__TransferFailed();
13         }
14         uint256 shares = tokenVault.deposit(s_guardianStakePrice, msg.
15             sender);
16         if (shares == 0) {
17             revert VaultGuardiansBase__TransferFailed();
18         }
19         return address(tokenVault);
20     }
```

according to the docs provided, the `VaultGuardianToken` purpose is to cast votes on proposals to: - Updating pricing parameters - Getting a cut of all performance of all guardians

But, there is no way for the guardian to `burn` or dispose of the `VaultGuardianToken` even after quitting the guardian role

Impact: As a result, the ex-guardian still have a power to vote on the DAO without any cost. anyone can become a guardian and quit multiple times to get more and more `VaultGuardianToken`.

Proof of Concept: Consider the following scenario:

1. A malicious user had enough money to become a guardian (whether through flashloans or their own wealth)
2. They become a guardian by paying the amount of `VaultGuardiansBase::s_guardianStakePrice`
3. Protocol mint `VaultGuardianToken` the same amount as `VaultGuardiansBase::s_guardianStakePrice`
4. After receiving the token, the malicious user quit becoming a guardian by calling `VaultGuardiansBase::quitGuardian`
5. Repeat step 2

Proof of Code

add the following test function into `test/unit/concrete/VaultGuardiansBaseTest.t.sol` test suite

```
1      function testGuardianCanMintInfiniteAmountOfVaultGuardianTokens()
2          public {
3              // 1. The malicious user have enough money
4              address badUser = makeAddr("malicious user");
5              uint256 badUsersCurrentWETH = 10 ether;
6              weth.mint(badUsersCurrentWETH, badUser);
7              allocationData = AllocationData(1000, 0, 0); // we allocate the
8                  data to all hold
9
10             // 2. The malicious user become a guardian
11             // 2.1 we made sure we know the current balance before and
12             // after becoming guardian
13             uint256 wethBalanceBefore = weth.balanceOf(address(badUser));
14
15             vm.startPrank(badUser);
16             // 2.2 the malicious user approve the the transfer of weth to
17             // vaultGuardians
18             weth.approve(address(vaultGuardians), badUsersCurrentWETH);
19
20             // 2.3 malicious users becoming guardian and receive
21             // VaultGuardianTokens
```

```
17     address wethVault = vaultGuardians.becomeGuardian(
18         allocationData);
19     VaultShares vaultSharesWeth = VaultShares(wethVault);
20     // 3. Bad users then quit the guardian role
21     // 3.1 we must approve the VaultGuardian contract to receive
22     // our shares
23     vaultSharesWeth.approve(address(vaultGuardians), type(uint256).
24         max);
25     // 3.2 malicious users quit becoming guardian
26     vaultGuardians.quitGuardian();
27     vm.stopPrank();
28     uint256 wethBalanceAfter = weth.balanceOf(address(badUser));
29     uint256 vaultGuardianTokenBalance = vaultGuardianToken.
30         balanceOf(badUser);
31     // 4. Made sure the balance of weth is the same before and
32     // after
33     assertApproxEqAbs(wethBalanceAfter, wethBalanceBefore, 1e16);
34     assertApproxEqAbs(wethBalanceAfter, vaultGuardianTokenBalance,
35         1e16);
36     // 5. we redo the second step over-and over again
37 }
```

Recommended Mitigation: The simplest method is to burn the `VaultGuardianToken` when a guardian wants to quit. But remember to burn the exact amount the guardian minted when becoming a guardian, not burning the same value as `VaultGuardiansBase::s_guardianStakePrice`, since the value can be changed less than the correct amount

[H-2] Miscalculation in `UniswapAdapter::_uniswapInvest` causes allocation data to be wrong

Description: According to the docs written for `UniswapAdapter::_uniswapInvest` function, its intend is to swap half the allocation value to receive the counter token pairs, and the other half to invest into Uniswap's liquidity pool to obtain liquidity tokens.

But after swapping was successful, they added the full amount of allocation value and the swapped counter token pairs as liquidity to Uniswap

```
1     function _uniswapInvest(IERC20 token, uint256 amount) internal {
2         .
3         .
4         .
5         @> uint256 amountOfTokenToSwap = amount / 2;
6         .
```

```

7      .
8      .
9      uint256[] memory amounts = i_uniswapRouter.
        swapExactTokensForTokens({
10         amountIn: amountOfTokenToSwap,
11         .
12         .
13         .
14         .
15         .
16 @>      succ = token.approve(address(i_uniswapRouter),
        amountOfTokenToSwap + amounts[0]);
17         .
18         .
19         .
20
21         (uint256 tokenAmount, uint256 counterPartyTokenAmount, uint256
        liquidity) = i_uniswapRouter.addLiquidity({
22         .
23         .
24         .
25 @>         amountADesired: amountOfTokenToSwap + amounts[0],
26         .
27         .
28         .
29     }

```

Impact: Because of this, the actual amount allocated to be invested in Uniswap was 1.5 times bigger than the intended amount

Proof of Concept:

1. supposed the Allocation data is 200, 400, 400 (meaning 20% hold, 40% Uniswap, 40% AAVE)
2. someone becomes a guardian and automatically deposit the money to the created vault. (for simplicity, assuming the first deposited amount was 10 ether, same as contract)
3. 40% of 10 ether is 4 ether allocated for investing in Uniswap
4. Calculation in Uniswap invest:
 - 4 ether / 2 = 2 ether `uint256 amountOfTokenToSwap = amount / 2;`
 - the other 2 is swapped becoming a 2 ether worth of whatever token it is, but for the sake of this example we used USDC [WETH spent : 2 ether]
 - The desired amount to be invested into Uniswap pool is `amountADesired: amountOfTokenToSwap + amounts[0]` -And since in the mock the `amounts[0]` is set as `amountIn` inside `UniswapRouterMock`

```

1  function swapExactTokensForTokens(
2      uint256 amountIn,

```



```
3      .
4      .
5      .
6  ) external returns (uint256[] memory amounts) {
7      .
8      .
9      .
10     amounts[0] = amountIn;
11     .
12     .
13     .
14 }
```

- amountADesired: 2 ether + 2 ether = 4 ether [WETH spent : 4 ether]

5. total WETH spent is 2 ether for swapping and 4 ether for adding liquidity, which adds up to actually spending 6 ether
6. since 40% (4 ether) of the allocation is spent for AAVE and 6 ether is spent for Uniswap, the remaining Hold balance will be 0

Proof of Code

add the following test function into `test/unit/concrete/VaultGuardiansBaseTest.t.sol` test suite

```
1  function
2      testAllocationDataIsNotAccurateBecauseUniswapMockMiscalculate()
3      public {
4      weth.mint(mintAmount, guardian);
5
6      vm.startPrank(guardian);
7      uint256 wethBalanceBefore = weth.balanceOf(address(guardian));
8      weth.approve(address(vaultGuardians), mintAmount);
9
10     // set the allocation data
11     allocationData = AllocationData(200, 400, 400);
12
13     address wethVault = vaultGuardians.becomeGuardian(
14         allocationData);
15     vm.stopPrank();
16
17     assertEq(weth.balanceOf(wethVault), 0);
18 }
```

Recommended Mitigation: Fix the mock, the real Uniswap did not make such mistake

[H-3] Ignoring slippage protection Uni swapAdapter :: _uniswapInvest function

Description: `UniswapAdapter::_uniswapInvest` function set the minimum amount to be received after swapping and after adding liquidity is 0

```
1     function _uniswapInvest(IERC20 token, uint256 amount) internal {
2         .
3         .
4         .
5         uint256[] memory amounts = i_uniswapRouter.
            swapExactTokensForTokens({
6             .
7             .
8             .
9 @>         amountOutMin: 0,
10            .
11            .
12            .
13
14            // amounts[1] should be the WETH amount we got back
15            (uint256 tokenAmount, uint256 counterPartyTokenAmount, uint256
                liquidity) = i_uniswapRouter.addLiquidity({
16                .
17                .
18                .
19 @>         amountAMin: 0,
20 @>         amountBMin: 0,
21            .
22            .
23            .
```

Impact: very bad practice, anyone can manipulate the price by frontrunning it and flash loaning the pool to change the price to be unfavorable for the transaction

Proof of Concept: no

Recommended Mitigation: add a dynamic value to be set by guardian

```
1 -     function _uniswapInvest(IERC20 token, uint256 amount) internal {
2 +     function _uniswapInvest(IERC20 token, uint256 amountOutForSwap,
            uint256 minAmountA, uint256 minAmountB) internal {
3         .
4         .
5         .
6         uint256[] memory amounts = i_uniswapRouter.
            swapExactTokensForTokens({
7             .
8             .
9             .
10 -         amountOutMin: 0,
```

```
11 +         amountOutMin: amountOutForSwap,
12         .
13         .
14         .
15
16         // amounts[1] should be the WETH amount we got back
17         (uint256 tokenAmount, uint256 counterPartyTokenAmount, uint256
18         liquidity) = i_uniswapRouter.addLiquidity({
19         .
20         .
21         .
22         .
23         .
24         .
25         .
26         .
27         .
```

because of this we must update the `VaultShares::_investFunds` and other related function for guardian to set the dynamic value

Medium

[M-1] `VaultShares::updateHoldingAllocation` did not update the holding allocations automatically

Description: `VaultShares::updateHoldingAllocation` only update the `VaultShares::s_allocationData` state variables and not actually updating the allocation immedietly

```
1     function updateHoldingAllocation(AllocationData memory
2         tokenAllocationData) public onlyVaultGuardians isActive {
3         uint256 totalAllocation = tokenAllocationData.holdAllocation +
4             tokenAllocationData.uniswapAllocation
5             + tokenAllocationData.aaveAllocation;
6         if (totalAllocation != ALLOCATION_PRECISION) {
7             revert VaultShares__AllocationNot100Percent(totalAllocation
8             );
9         }
10        s_allocationData = tokenAllocationData;
11        emit UpdatedAllocation(tokenAllocationData);
12    }
```

Impact: guardians or anyone need to call `VaultShares::rebalanceFunds` manually in order to made sure that all the funds are correctly allocated

Proof of Concept:

Consider the following scenario 1. Guardian update the allocation by calling `VaultShares::updateHoldingAllocation` 2. The `VaultShares::s_allocationData` did update it but it did not relocate the actual funds 3. someone have to call `VaultShares::rebalanceFunds` to actually relocate the funds to the correct planned allocation

Recommended Mitigation:

1. Use the `divestThenInvest` modifier on the function

```
1 - function updateHoldingAllocation(AllocationData memory
   tokenAllocationData) public onlyVaultGuardians isActive {
2 + function updateHoldingAllocation(AllocationData memory
   tokenAllocationData) public divestThenInvest onlyVaultGuardians
   isActive {
3     .
4     .
5     .
```

2. Or just call `VaultShares::rebalanceFunds` at the end of the function

```
1     function updateHoldingAllocation(AllocationData memory
   tokenAllocationData) public onlyVaultGuardians isActive {
2         uint256 totalAllocation = tokenAllocationData.holdAllocation +
   tokenAllocationData.uniswapAllocation
3         + tokenAllocationData.aaveAllocation;
4         if (totalAllocation != ALLOCATION_PRECISION) {
5             revert VaultShares__AllocationNot100Percent(totalAllocation
   );
6         }
7         s_allocationData = tokenAllocationData;
8         emit UpdatedAllocation(tokenAllocationData);
9 +     rebalanceFunds();
10 }
```

Questions

What is the purpose of minting shares for `VaultGuardians` contact everytime someone calls `VaultShares::deposit` function?

```
1     function deposit(uint256 assets, address receiver)
2         public
3         override(ERC4626, IERC4626)
4         isActive
5         nonReentrant
6         returns (uint256)
7     {
```

```
8      if (assets > maxDeposit(receiver)) {
9          revert VaultShares__DepositMoreThanMax(assets, maxDeposit(
            receiver));
10     }
11
12     uint256 shares = previewDeposit(assets);
13     console2.log("VAULT SHARES BEFORE DEPOSIT", IERC20(asset()).
        balanceOf(address(this)));
14
15     _deposit(_msgSender(), receiver, assets, shares);
16     console2.log("VAULT SHARES AFTER DEPOSIT", IERC20(asset()).
        balanceOf(address(this)));
17
18     _mint(i_guardian, shares / i_guardianAndDaoCut);
19 @>   _mint(i_vaultGuardians, shares / i_guardianAndDaoCut); // what
      for ??
20
21     _investFunds(assets);
22     return shares;
23 }
```